

Periodicity Checking: Skipping the Interior Grind

Matias Saavedra

Friday 10th July, 2026

Binary: branch feat/periodicity-check, commit fa6d8ac (tag binary-v6-periodicity), built on top of main (3061307, the binary-v5-affinity lineage). Baseline is that main commit itself. Raw data: experiments/21-periodicity-check/.

Resumen

This report implements and measures the work-reduction lever that every report since 09-9point-sampling.tex has pointed at: periodicity checking in the CPU `diverge()`. A Brent-style cycle detector saves the orbit state at doubling intervals and returns `MAXITER` as soon as the floating-point state exactly revisits the saved state — an exactly-repeating orbit of a deterministic map can never escape, so the early return is provably exact. Measured on the production spec (100 frames, 1920×1080 , `diffT` = 0.1, `save` = 1, 3 alternating reps, on AC): CPU12 falls $164.4 \rightarrow 35.2$ s (-78.6% , $4.67\times$) and the best hybrid falls $57.6 \rightarrow 30.9$ s (-46.3% , $1.86\times$) — a larger win than every scheduling lever combined (affinity -3.6% , report 12; second GPU -23.2% , report 20), from 18 added lines in one function. The mechanism is work elimination: the CPU pool’s compute drops $1,917 \rightarrow 402$ s while the 5,608-leaf decomposition and all 100 output frames stay byte-identical. The hybrid queue self-rebalances — CPU threads, now $\sim 5\times$ faster on interior content, take 4,279 regions (was 2,160) and the GPU keeps only the 1,329 largest. The CPU pool remains the binding resource (busiest thread 29.5 s $\approx$ wall), but what is left on it is detection latency plus the genuinely-escaping boundary band, which no exact early-out can skip.

1. Motivation: 99.5% of the iterations prove what a cycle detector knows early

Report 09 established that splitting conserves total compute (~ 495 G iterations regardless of decomposition) and report 12 left the system with both executors near saturation — scheduling was exhausted. What no report had quantified is what the iterations are for. Parsing the per-region work metrics of the report-12 production run (`experiments/12-gpu-affinity/logs/dist_aff.stderr`: all 5,608 leaves with size, mean iterations and interior fraction) gives Table 1.

Tabla 1: Leaf census of the production run at `diffT = 0.1`, bucketed by interior fraction (`inset`, the fraction of pixels that reach the frame’s `MAXITER`). Work is total executed iterations. Read the last column top to bottom: exterior content is free, interior content is nearly everything.

interior fraction	leaves	% of pixels	% of work
0% (pure exterior)	2,123	33.2	0.09
0–1%	292	4.6	0.06
1–25%	621	9.7	2.29
25–75%	870	13.6	13.91
75–< 100%	784	14.5	25.18
100% (pure interior)	918	24.5	58.46

Weighting each leaf’s interior pixels by the frame’s `MAXITER` shows that 99.5% of all 495 G iterations are spent on interior pixels — pixels whose orbits never escape, grinding to the iteration cap only to confirm it. The entire boundary band plus all exterior costs 0.5%. The wall itself is the CPU pool grinding that content: in the report-12 run the 11 CPU threads spend 545 s of compute, 99.0% of it on interior pixels, at 252 M iterations/s/thread — content the GPU runs at 7.9 G iterations/s (31×) but cannot absorb (kernel already 91% busy, report 12). Interior pixels are exactly the ones whose orbits settle into an attracting cycle, so a cycle detector can return the same answer after the transient dies out instead of after `MAXITER` iterations. That is the lever implemented here.

2. The change: an exact Brent cycle detector

`MandelRegion::diverge()` gains a saved orbit state (s_x, s_y) , refreshed at doubling intervals (iterations 8, 16, 32, ...); every iteration compares the new state against it with exact floating-point equality:

Código 1: `mandelregion.cpp` — the modified `diverge()` inner loop (commit `fa6d8ac`).

```

1 double sx = vx, sy = vy;
2 int nextSave = 8;
3 while (iter < MAXITER && (vx * vx + vy * vy) < 4)
4 {
5     tx = vx * vx - vy * vy + cx;
6     ty = 2 * vx * vy + cy;
7     vx = tx;
8     vy = ty;
9     iter++;
10    if (vx == sx && vy == sy)
11        return MAXITER;           // exact cycle: the orbit can never escape
12    if (iter == nextSave)
13    {
14        sx = vx;                   // Brent: refresh the anchor at iterations
15        sy = vy;                   // 8, 16, 32, ... so a period-p cycle entered

```

```

16     nextSave *= 2;           // after a transient t is caught in  $O(\max(p,t))$ 
17 }
18 }
19 return iter;

```

Why this is exact.

The iteration is a deterministic map on the floating-point numbers: the successor of (v_x, v_y) is a pure function of (v_x, v_y, c_x, c_y) . If the state ever equals a previously visited state, the orbit is in a cycle of the FP map and will repeat forever — the escape test can never fire, so the unmodified loop would have ground to **MAXITER** and returned **MAXITER**. Returning it at detection time changes when the answer arrives, never what it is. Conversely, an orbit that eventually escapes can never exactly revisit a state (it would otherwise cycle and not escape), so no escaping pixel can be misclassified: there are no false positives by construction, not by tolerance tuning. This is why exact equality is used instead of an epsilon — an epsilon comparison would detect sooner but could constant-fill slowly-diverging boundary orbits, changing the image. Section 4.4 confirms the guarantee empirically: all 100 frames byte-identical, decomposition unchanged.

Detection latency.

Brent’s doubling schedule catches a cycle of period p entered after a transient of t iterations within $O(\max(p, t))$: once the anchor lands inside the cycle and the window $\geq p$, the comparison must hit. Figure 1 traces the detector on an interior point of the period-3 bulb: the FP orbit converges geometrically to the attracting 3-cycle, the distance-to-anchor sawtooth collapses by ~ 3 decades per checkpoint window, and at iteration 67 the state is bitwise equal to the anchor — 67 iterations instead of the frame’s thousands.

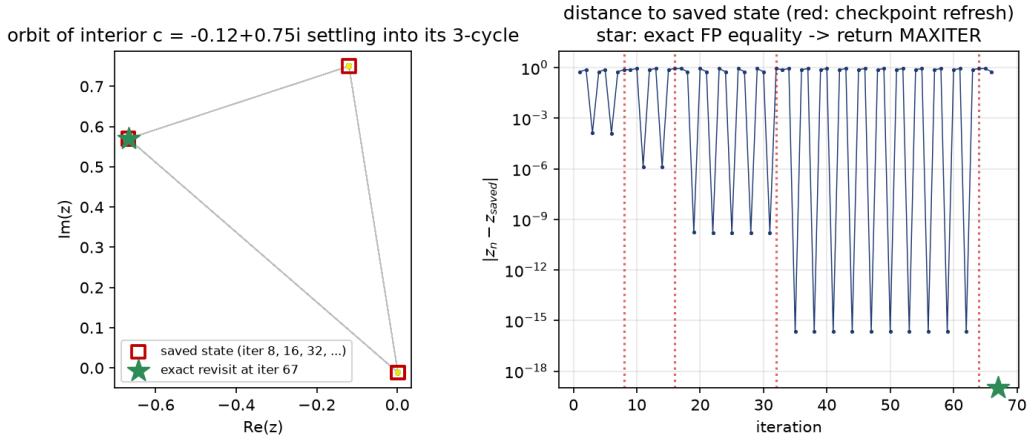


Figure 1: The detector on $c = -0.12 + 0.75i$ (period-3 bulb interior). Left: the orbit in the z -plane (colour = iteration) hops between the three limbs of its attracting cycle; red squares are the Brent anchors (iterations 8, 16, 32, 64), the green star the exact FP revisit at iteration 67. Right: distance from the current state to the anchor (log scale); each dotted red line is an anchor refresh, and the geometric collapse ends in exact equality (star) — the loop returns `MAXITER` there, skipping every remaining iteration.

Cost.

The check adds two compares and a predictable branch to a loop of ~ 10 FP operations, paid by every iteration, including escaping pixels that never cycle. This bounds the worst case: exterior and boundary-band pixels carry 0.5% of all iterations ($\mathcal{S}1$), so even a 30% per-iteration slowdown on that slice is $< 0.2\%$ of wall — invisible next to the interior savings.

CPU branch only.

Report 16 measured the GPU’s interior kernels at exactly 100.0% warp-execution efficiency and 85.6% of FP64 peak: pixels in a warp bail out at different iterations under periodicity checking, so the same change on the GPU would introduce divergence into a perfectly-coherent FP64-bound kernel. `kernel.cu` is untouched; a GPU variant is a separate A/B (recommendation 3).

3. Methodology

- **Hardware/software:** i7-9750H (6C/12T) + GTX 1660 Ti Max-Q, same laptop as all prior reports; on AC power, rebooted before the session.
- **Binaries:** baseline built from `main` (3061307) in a clean git worktree; treatment from `fa6d8ac` (= `main` + the `diverge()` change only — an 18-line diff in one function).
- **Workload:** production `spec.in` — 100 frames, 1920×1080 , deep zoom to `maxIter` 10,000; `diffT` = 0.1, `pixT` = 32,768, `quiet` = 1, `save` = 1 (the Fig. 11.15 methodology, same as the experiment-20 headline).

- **Configs:** CPU12 (mode 0; the change carries everything) and dGPU+11CPU (mode 1; the production config). 3 reps each, binaries alternated inside every rep so thermal drift hits both sides equally (`experiments/21-periodicity-check/ab.sh`).
- **Calibration:** the baseline reproduces its references — CPU12 164.4 s vs. experiment 20’s 161.6 s (+1.7%, third rep warmest); hybrid 57.6 s vs. 57.7 s (−0.1%). The A/B is internally consistent regardless.

Incidental finding: spec-prefix buffer overflow.

Smoke-testing with a spec whose image prefix was an absolute scratch path produced garbage timings in both binaries: `main.cpp` reads the prefix with `fin >> imageFilePrefix` into `char[MAXFNAME-8]` (42 bytes), so any prefix over 41 characters silently smashes the stack — observed as `maxIterations[]` corrupted to 24 in one binary and to a huge value in the other (layout-dependent). Production specs use the 3-character prefix `img`, so no historical measurement is affected. All runs in this report use short prefixes; a bounds-checked read should land as its own change on `main`.

4. Results

4.1. End-to-end wall

Tabla 2: End-to-end wall (s), 3 reps and mean $\pm$ sample σ . Negative Δ means the periodicity check helped. Every treatment rep beats every baseline rep by >26 s in both configs; at these σ the deltas are $>40\sigma$ — unambiguous signal.

config	main	periodicity	Δ	speed-up
CPU12 (mode 0)	164.43 ± 2.10	35.21 ± 0.17	−78.6%	$4.67\times$
dGPU+11CPU (mode 1)	57.64 ± 0.45	30.94 ± 0.56	−46.3%	$1.86\times$

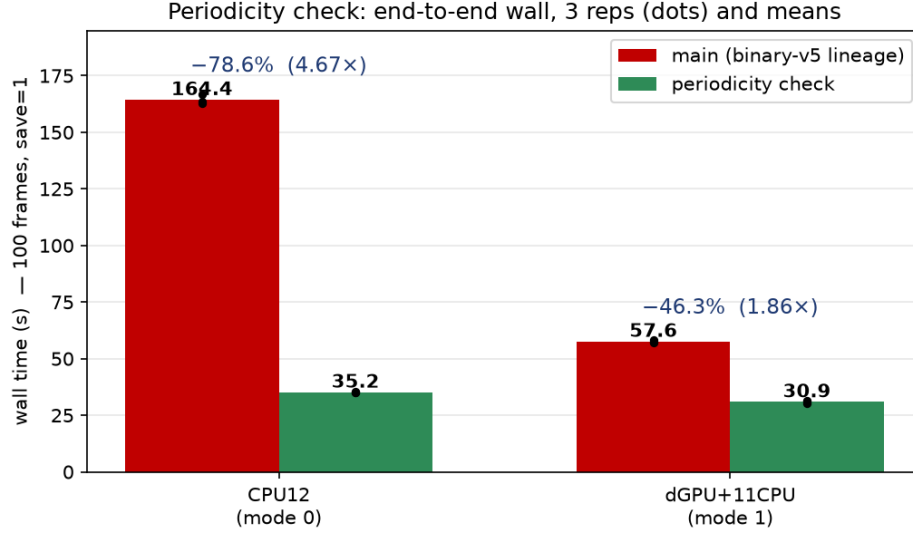


Figure 2: Wall time per config (bars: means; dots: the 3 individual reps). Red is the **main** baseline, green the periodicity binary. 30.9 s is the new best wall for this workload on this machine — below the two-GPU mode-3 result of report 20 (44.3 s at `save=1`) using one GPU.

For scale: GPU affinity moved the wall -3.6% (report 12), adding the integrated GPU as a third executor moved it -23.2% (report 20). This single CPU-side change moves it -46.3% .

4.2. Where the time went

Tabla 3: Executor accounting in hybrid mode 1 (means over 3 reps, from the per-run summaries). The decomposition is fixed at 5,608 regions; only who computes them and how long changes. “Pool” sums the 11 CPU threads’ compute time.

	main	periodicity	Δ
GPU regions	3,448	1,329	-61%
CPU regions	2,160	4,279	$+98\%$
GPU kernel total (s)	43.0	25.7	-40%
GPU avg per region (ms)	12.4	19.6	$+58\%$
CPU pool compute (s)	620.9	324.2	-48%
CPU busiest thread (s)	56.8	29.7	-48%

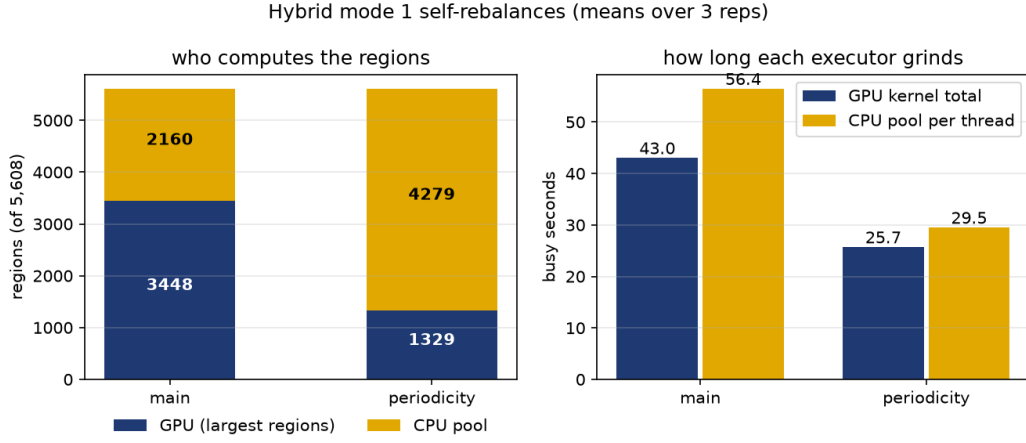


Figure 3: The rebalance at a glance. Left: the CPU pool takes over most of the region count (4,279 of 5,608) because its threads are now cheap on interior content; the GPU keeps only the largest regions (its per-region average rises 12.4 to 19.6 ms). Right: both executors’ busy time drops and stays nearly matched (25.7 s kernel vs. 29.5 s per CPU thread) — the queue re-found its balance around the new cost structure with no code change.

In CPU-only mode the accounting is starker: pool compute falls $1,917 \rightarrow 402$ s (-79%) — matching the $4.67\times$ wall ratio, i.e. the win is work elimination at fixed balance, not better scheduling.

4.3. Per-region cost distribution

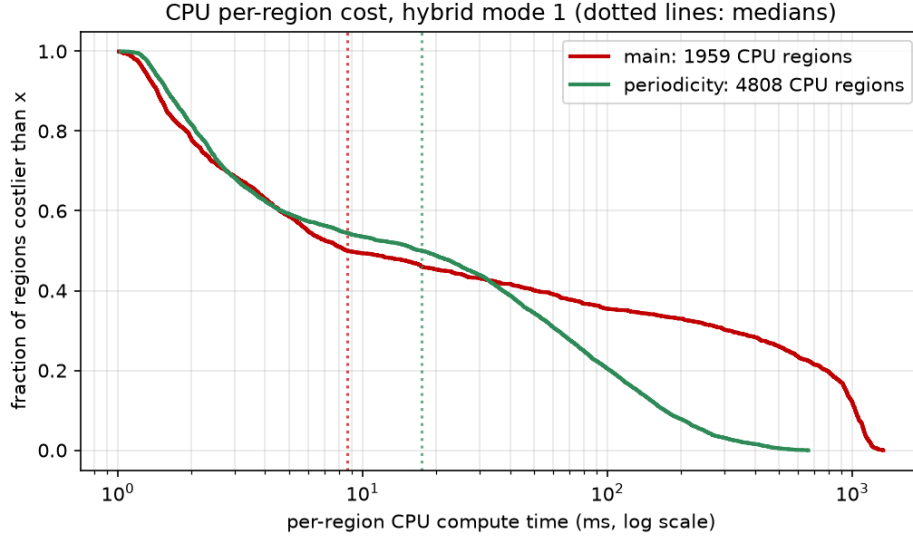


Figure 4: Survival function of per-region CPU compute time in hybrid mode 1 (log x ; fraction of regions costlier than x). The green (periodicity) population is larger — 4,808 regions vs. 1,959, because the faster CPU pool now takes regions the GPU used to — yet its heavy tail collapses: the worst CPU region halves ($1,340 \rightarrow 662$ ms) and the >100 ms tail thins from 34% of regions to 8%. The crossing at ~ 30 ms is the signature of taking over more but cheaper-per-pixel work.

The distributions come from the `quiet=0` runs (`dist_aff.stderr` of experiment 12 for the baseline; `logs/dist_pc.stderr` here). Median rises $8.7 \rightarrow 17.5$ ms (the pool absorbs formerly-GPU mid-size regions), total falls $545 \rightarrow 284$ s.

4.4. Output and decomposition verification

- **Byte identity:** the full spec rendered CPU-only (mode 0 — every pixel through the modified code path) by both binaries; `cmp` on all frames: 100/100 byte-identical (`verify_identity.sh`; mode-0 pixel values are independent of thread count and decomposition, so the comparison is exact). The GPU path is untouched by the branch, so this covers the entire change.
- **Decomposition:** all 12 A/B runs report exactly 5,608 leaves — the known `diffT=0.1` decomposition (reports 09/18). The check returns identical values at the 9-point stencil samples, so `examine()` takes identical decisions.

5. Analysis

5.1. The win is work elimination, and it lands exactly where $\mathcal{S}1$ said the work was

CPU12 pool compute drops $1,917 \rightarrow 402$ s (-79%), against the 99.5% interior-work share measured in $\mathcal{S}1$. The reason the two numbers agree is direct: every interior pixel used to cost the frame’s MAXITER (up to 9,901) iterations; with the detector it costs the transient-plus-period of its orbit. The residual 402 s is the boundary band (escaping pixels cannot be skipped — they never cycle) plus detection latency. The implication closes the arc of reports 09–15: after scheduling was exhausted (affinity -3.6% , shared queue ± 0 , async streams priced at $<1\%$), the first work lever is worth $12\times$ more than the best scheduling lever.

5.2. Detection latency on the production content is ~ 900 iterations-equivalent, not 67

The single-region micro-benchmark (the zoom target rendered as one pure-interior 1920×1080 region on one thread) runs $67.27 \rightarrow 5.98$ s ($11.2\times$). At the measured 308 M iterations/s that is ~ 890 iterations-equivalent per pixel — far above the 67 of Figure 1’s period-3 point. This is consistent with the content: the production zoom terminates inside a deep minibrot (report 08), whose attracting cycles have high periods and long transients, and Brent’s schedule pays $O(\max(p, t))$ with up to a $2\times$ overshoot from the doubling window. The implication: the speed-up is content-dependent — shallow interior (main cardioid, low-period bulbs; the report-13 **inside** and **seahorse** regimes) will detect in tens of iterations, while deep minibrot interior keeps hundreds — but it is bounded below by the exactness guarantee: the detector never does worse than the grind by more than the 2-compare overhead.

5.3. The hybrid self-rebalances; the CPU pool remains the (much lower) wall

No scheduling code changed, yet the GPU’s share fell from 3,448 to 1,329 regions (Table 3). The reason is the min-max queue of report 12 reacting to the new cost structure: CPU threads drain the small end $\sim 5\times$ faster, reach deeper into the queue, and leave the GPU only the regions where it still wins big (its average region grows $12.4 \rightarrow 19.6$ ms — it keeps the largest). Busy times land near-matched: 25.7 s of kernel vs. 29.5 s per CPU thread against a 30.9 s wall — the CPU pool is still the binding resource, but barely, and the balance re-found itself. The implication for report 20: the iGPU’s -23.2% was priced against a CPU pool grinding interior at full cost; with that grind mostly gone, the marginal value of a second GPU lane must be re-measured before merging `feat/igpu-opencl` (recommendation 2).

5.4. What the check cannot touch, and what that bounds

Three costs remain by construction: (i) escaping pixels — the boundary band — which never cycle and now dominate the CPU pool’s residual 402 s (CPU-only); (ii) detection latency, reducible only by a cheaper or earlier-firing detector (an epsilon tolerance — rejected here because it forfeits the exactness guarantee); (iii) the per-iteration check overhead, bounded at $<0.2\%$ of wall by the 0.5% work share of non-interior content. Not measured: an exterior-dominated workload (the report-13 **out-**

side regime) where the check only costs; the bound says the cost is the two-compare overhead on cheap pixels, but the honest statement is that this report measured the production zoom and its interior-heavy relatives only.

6. Recommendations

1. Merge `feat/periodicity-check` to `main` and tag `binary-v6-periodicity`. It is the new best binary: hybrid $57.6 \rightarrow 30.9$ s (-46.3%), CPU12 $164.4 \rightarrow 35.2$ s (-78.6%), output byte-identical, 18 lines in one function. Trade-off: none measured — the worst case is the $<0.2\%$ overhead bound on exterior content.
2. Re-run the experiment-20 A/B on top of v6 before merging `feat/igpu-openc1`. The iGPU’s -23.2% was measured against a CPU pool that no longer exists ($620 \rightarrow 324$ s of grind). Expected outcome: the mode-3 win shrinks substantially — the serial tail of big interior regions that motivated the second lane is now $5\times$ cheaper to clear; whether it still pays for displacing a CPU worker is an open measurement. Cost: one ~ 15 -minute sweep.
3. A/B a GPU-side periodicity variant separately — expect a loss. Report 16 measured the interior kernels at 100.0% warp efficiency and 85.6% of FP64 peak; per-pixel early-out would introduce divergence into that kernel and add FP64 compare work to the bottleneck pipe. Measure once for the record; do not merge blind.
4. Fix the spec-prefix overflow on `main`. Replace the unbounded `fin >> image-FilePrefix` with a bounded read (`std::string` + length check). Zero performance impact; removes a silent-corruption trap that cost this session its first smoke measurements.
5. Re-price the experiment-19 threshold sweep on v6. The partial sweep’s preliminary optimum ($\text{pixT} = 131,072$) was driven by per-region overhead against interior grind; with interior $5\times$ cheaper on the CPU, the optimum may move again. Cost: the remaining 55 runs, now $\sim 3\times$ faster on the treatment side.

7. Conclusion

An 18-line exact cycle detector in the CPU `diverge()` — anchor state refreshed at doubling intervals, return `MAXITER` on exact FP revisit — eliminates most of the interior grind that `S1` showed to be 99.5% of all iterations: CPU12 -78.6% (164.4 to 35.2 s), hybrid -46.3% (57.6 to 30.9 s, the new best wall), with all 100 frames byte-identical and the 5,608-leaf decomposition unchanged — the exactness argument (an exactly-repeating orbit cannot escape; an escaping orbit cannot exactly repeat) held empirically. The win is work elimination at fixed balance: pool compute -79% , the min-max queue re-balances itself around the new costs, and the CPU pool remains the binding resource at a quarter of its former cost. What remains is the truly-escaping boundary band, the content-dependent detection latency (~ 900 iterations-equivalent inside the deep minibrot vs. 67 in a period-3 bulb), and two follow-up measurements this result re-opens: the iGPU’s marginal value (report 20’s baseline no longer exists) and the threshold-sweep optimum.